

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**

Факультет информационных технологий
Кафедра информатики и информационных технологий
Направление подготовки 09.03.02
«Информационные системы и технологии»

ЛАБОРАТОРНАЯ РАБОТА № 2

Дисциплина: «Функциональное программирование»

Тема: «Применение принципов функционального программирования
при разработке веб-приложения „Список задач“»

Выполнила: студентка группы 241-335

Басимова Анастасия Маратовна

Дата, подпись _____ / _____

Проверил: _____

Дата, подпись _____ / _____

Замечания: _____

Москва, 2026

1 ЦЕЛЬ РАБОТЫ

Применить принципы функционального программирования для разработки небольшого веб-приложения.

2 ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

Разработать веб-приложение «Список задач», которое позволяет пользователю:

- добавлять новые задачи;
- отмечать задачи как выполненные;
- удалять задачи;
- фильтровать задачи по статусу (выполненные / невыполненные).

Требования к реализации:

- использовать принципы функционального программирования, такие как иммутабельность данных и чистые функции;
- использовать функции высшего порядка для обработки списка задач;
- веб-приложение должно быть реализовано с использованием HTML, CSS и JavaScript;
- интерфейс должен быть интуитивно понятным и удобным для пользователя.

3 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Иммутабельность (неизменяемость) данных означает, что существующие структуры данных не модифицируются: вместо изменения массива или объекта создаётся его новая версия с нужными правками. В JavaScript для этого применяются оператор расширения, методы `map` и `filter`, возвращающие новый массив, а также метод `Object.freeze`, запрещающий изменение свойств объекта.

Чистая функция при одинаковых аргументах всегда возвращает одинаковый результат и не имеет побочных эффектов. Работа с DOM, обработка событий и вывод на экран побочными эффектами являются, поэтому в приложении они вынесены в отдельный слой, а вся обработка данных выполняется чистыми функциями.

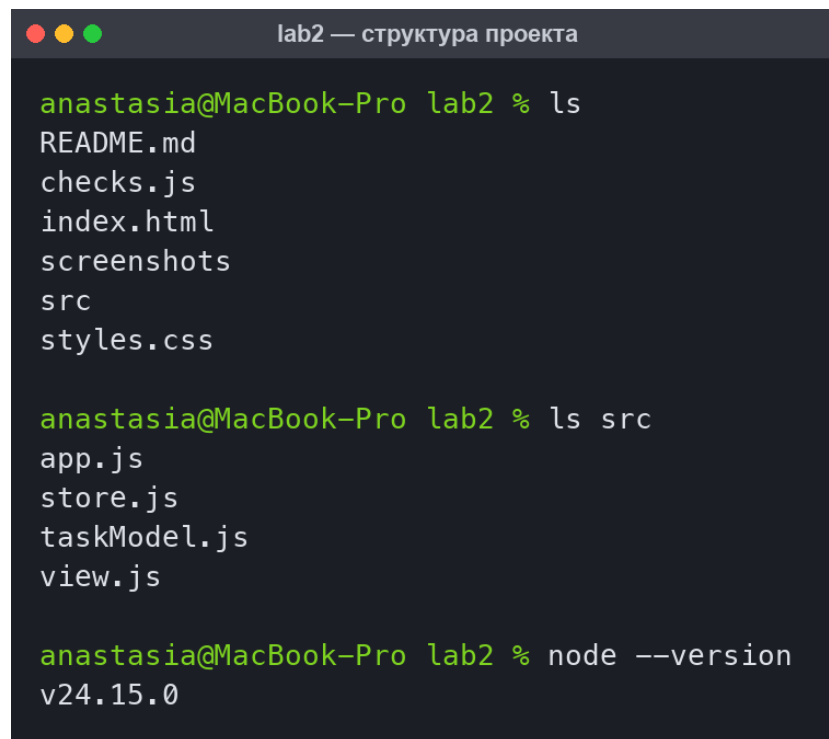
Функции высшего порядка принимают другие функции в качестве аргументов или возвращают их как результат. В приложении такими функциями являются встроенные `map`, `filter`, `reduce` и `forEach`, а также функция создания хранилища `createStore`, принимающая функцию перехода состояний, и функция подписки `subscribe`, принимающая обработчик изменений.

Состояние приложения описывается одним неизменяемым объектом, а его изменение выполняется чистой функцией перехода `reducer` вида (состояние, действие) → новое состояние. Такой подход, известный по библиотеке `Redux`, сводит всю изменяемую часть программы к одной переменной внутри хранилища и делает поведение приложения предсказуемым.

4 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

4.1 Структура проекта

Приложение реализовано средствами `HTML`, `CSS` и `JavaScript` без использования внешних библиотек и запускается открытием файла `index.html` в браузере. Логика разделена на четыре модуля: модель данных, хранилище состояния, представление и связывающий их слой работы с DOM. Отдельный файл `checks.js` предназначен для проверки чистоты функций модели средствами `Node.js`. Структура проекта приведена на рисунке 1.



```
lab2 — структура проекта

anastasia@MacBook-Pro lab2 % ls
README.md
checks.js
index.html
screenshots
src
styles.css

anastasia@MacBook-Pro lab2 % ls src
app.js
store.js
taskModel.js
view.js

anastasia@MacBook-Pro lab2 % node --version
v24.15.0
```

Рисунок 1 – Структура проекта

4.2 Модель данных: чистые функции

Модуль `src/taskModel.js` содержит чистые функции работы со списком задач. Функция `addTask` возвращает новый массив с добавленной задачей, `toggleTask` строит новый массив методом `map`, инвертируя статус только у нужной задачи, `removeTask` отбирает элементы методом `filter`, а `filterTasks` выполняет отбор по статусу. Каждая задача создаётся методом `Object.freeze`, поэтому изменить её свойства напрямую невозможно.

Предикаты фильтрации хранятся в объекте `predicates`, то есть функции используются как обычные данные: нужный предикат выбирается по ключу и передаётся методу `filter`. Статистика по списку вычисляется функцией `countTasks` через `reduce`, что также не изменяет исходный массив.

Листинг 1 – Модуль `src/taskModel.js`

```
/**
 * Модель данных: чистые функции для работы со списком задач.
 * Ни одна функция не изменяет переданные аргументы —
 * каждая возвращает новое значение.
```

```

*/
const TaskModel = (() => {
  /** Проверка корректности названия задачи. */
  const isValidTitle = (title) =>
    typeof title === 'string' && title.trim().length > 0;

  /** Следующий свободный идентификатор. */
  const nextId = (tasks) =>
    tasks.reduce((max, task) => Math.max(max, task.id), 0) + 1;

  /** Создание новой задачи (объект защищён от изменения). */
  const createTask = (id, title) =>
    Object.freeze({ id, title: title.trim(), done: false });

  /** Новый список с добавленной задачей. */
  const addTask = (tasks, title) =>
    isValidTitle(title)
      ? [...tasks, createTask(nextId(tasks), title)]
      : tasks;

  /** Новый список, в котором у задачи id инвертирован статус. */
  const toggleTask = (tasks, id) =>
    tasks.map((task) =>
      task.id === id ? Object.freeze({ ...task, done: !task.done }) : task,
    );

  /** Новый список без задачи с указанным id. */
  const removeTask = (tasks, id) => tasks.filter((task) => task.id !== id);

  /** Предикаты фильтрации: функции как данные. */
  const predicates = Object.freeze({
    all: () => true,
    active: (task) => !task.done,
    done: (task) => task.done,
  });

  /** Новый список, отобранный по статусу задач. */
  const filterTasks = (tasks, filter) =>
    tasks.filter(predicates[filter] ?? predicates.all);

  /** Статистика по списку: всего, выполнено, осталось. */
  const countTasks = (tasks) =>
    tasks.reduce(
      (acc, task) => ({
        total: acc.total + 1,
        done: acc.done + (task.done ? 1 : 0),
        active: acc.active + (task.done ? 0 : 1),
      }),
      { total: 0, done: 0, active: 0 },
    );

  return Object.freeze({
    isValidTitle,
    nextId,
    createTask,
    addTask,
  });
}

```

```

    toggleTask,
    removeTask,
    filterTasks,
    countTasks,
  });
}) ();

// Экспорт для запуска проверок в Node.js (в браузере не используется).
if (typeof module !== 'undefined' && module.exports) {
  module.exports = TaskModel;
}

```

4.3 Хранилище состояния и функция перехода

Модуль `src/store.js` описывает состояние приложения одним объектом, содержащим список задач и текущий фильтр. Чистая функция `reducer` получает текущее состояние и объект действия, а возвращает новое состояние, собранное оператором расширения, — исходное состояние при этом не изменяется.

Функция высшего порядка `createStore` принимает функцию перехода и начальное состояние, а возвращает набор функций для чтения состояния, отправки действия и подписки на изменения. Изменяемая переменная состояния существует в единственном месте программы — внутри этой функции.

Листинг 2 – Модуль `src/store.js`

```

/**
 * Хранилище состояния приложения.
 * Переход между состояниями описывает чистая функция reducer,
 * а изменяемая ссылка на текущее состояние существует
 * в единственном месте программы – внутри createStore.
 */
const Store = (() => {
  /** Начальное состояние приложения. */
  const initialState = Object.freeze({
    tasks: [],
    filter: 'all',
  });

  /**
   * Чистая функция перехода: (состояние, действие) -> новое состояние.
   * Исходное состояние не изменяется.
   */
  const reducer = (state, action) => {
    switch (action.type) {
      case 'ADD':
        return {
          ...state,
          tasks: TaskModel.addTask(state.tasks, action.title),

```

```

    };
    case 'TOGGLE':
      return {
        ...state,
        tasks: TaskModel.toggleTask(state.tasks, action.id),
      };
    case 'REMOVE':
      return {
        ...state,
        tasks: TaskModel.removeTask(state.tasks, action.id),
      };
    case 'FILTER':
      return { ...state, filter: action.filter };
    default:
      return state;
  }
};

/**
 * Создаёт хранилище: возвращает функции чтения состояния,
 * отправки действия и подписки на изменения.
 */
const createStore = (reduce, preloadedState) => {
  let state = preloadedState;
  const listeners = [];

  const getState = () => state;

  const subscribe = (listener) => {
    listeners.push(listener);
    return () => listeners.splice(listeners.indexOf(listener), 1);
  };

  const dispatch = (action) => {
    state = reduce(state, action);
    listeners.forEach((listener) => listener(state));
    return state;
  };

  return Object.freeze({ getState, dispatch, subscribe });
};

return Object.freeze({ initialState, reducer, createStore });
})();

if (typeof module !== 'undefined' && module.exports) {
  module.exports = Store;
}

```

4.4 Представление

Модуль `src/view.js` содержит чистые функции построения разметки: `taskHTML` формирует разметку одной задачи, `listHTML` собирает список

методами `map` и `join`, `counterText` возвращает текст счётчика, а `escapeHTML` экранирует символы, опасные для разметки. Единственная функция с побочным эффектом — `render`, которая записывает готовую разметку в DOM.

Листинг 3 – Модуль `src/view.js`

```
/**
 * Представление: чистые функции построения разметки
 * и единственная функция с побочным эффектом – render.
 */
const View = (() => {
  /** Экранирование символов, опасных для разметки. */
  const escapeHTML = (text) =>
    text.replace(
      /[&<>"']/g,
      (char) =>
        ({
          '&': '&amp;',
          '<': '&lt;',
          '>': '&gt;',
          '"': '&quot;',
          "'": '&#39;',
        })[char],
    );

  /** Разметка одной задачи. */
  const taskHTML = (task) => `
    <li class="task ${task.done ? 'task--done' : ''}" data-id="${task.id}">
      <label class="task__label">
        <input type="checkbox" class="task__check" ${
          task.done ? 'checked' : ''
        } data-action="toggle">
        <span class="task__title">${escapeHTML(task.title)}</span>
      </label>
      <button class="task__remove" data-action="remove" title="Удалить">
        &times;
      </button>
    </li>`;

  /** Разметка всего списка либо сообщения о пустом списке. */
  const listHTML = (tasks, filter) =>
    tasks.length > 0
      ? tasks.map(taskHTML).join('')
      : `<li class="empty">${emptyText(filter)}</li>`;

  /** Текст для пустого списка в зависимости от фильтра. */
  const emptyText = (filter) =>
    ({
      all: 'Список пуст. Добавьте первую задачу.',
      active: 'Невыполненных задач нет.',
      done: 'Выполненных задач пока нет.',
    })[filter] ?? 'Список пуст.';

  /** Текст счётчика задач. */
```

```

const counterText = (counts) =>
  `Всего: ${counts.total} · Осталось: ${counts.active} ` +
  ` · Выполнено: ${counts.done}`;

/** Побочный эффект: отображение состояния в DOM. */
const render = (elements, state) => {
  const visible = TaskModel.filterTasks(state.tasks, state.filter);
  const counts = TaskModel.countTasks(state.tasks);

  elements.list.innerHTML = listHTML(visible, state.filter);
  elements.counter.textContent = counterText(counts);

  elements.filters.forEach((button) => {
    const isActive = button.dataset.filter === state.filter;
    button.classList.toggle('filter--active', isActive);
  });
};

return Object.freeze({
  escapeHTML,
  taskHTML,
  listHTML,
  emptyText,
  counterText,
  render,
});
})();

if (typeof module !== 'undefined' && module.exports) {
  module.exports = View;
}

```

4.5 Связывание модели и интерфейса

Модуль `src/app.js` получает ссылки на элементы страницы, создаёт хранилище и подписывает на его изменения функцию отрисовки. Обработчики событий не изменяют данные самостоятельно: они лишь отправляют в хранилище описание произошедшего действия. Обработка нажатий на элементы списка выполнена делегированием — один обработчик на весь список определяет действие по атрибуту `data-action`.

Листинг 4 – Модуль `src/app.js`

```

/**
 * Связывание модели, хранилища и представления.
 * Здесь сосредоточена работа с DOM и событиями,
 * вся логика обработки данных вынесена в чистые функции.
 */
(() => {

```

```

const elements = {
  form: document.querySelector('#task-form'),
  input: document.querySelector('#task-input'),
  list: document.querySelector('#task-list'),
  counter: document.querySelector('#counter'),
  filters: Array.from(document.querySelectorAll('[data-filter]')),
};

const store = Store.createStore(Store.reducer, Store.initialState);

store.subscribe((state) => View.render(elements, state));

elements.form.addEventListener('submit', (event) => {
  event.preventDefault();

  if (!TaskModel.isValidTitle(elements.input.value)) {
    return;
  }

  store.dispatch({ type: 'ADD', title: elements.input.value });
  elements.input.value = '';
  elements.input.focus();
});

elements.list.addEventListener('click', (event) => {
  const action = event.target.dataset.action;
  const item = event.target.closest('.task');

  if (action === undefined || item === null) {
    return;
  }

  const id = Number(item.dataset.id);
  store.dispatch({ type: action === 'toggle' ? 'TOGGLE' : 'REMOVE', id });
});

elements.filters.forEach((button) => {
  button.addEventListener('click', () => {
    store.dispatch({ type: 'FILTER', filter: button.dataset.filter });
  });
});

View.render(elements, store.getState());
})();

```

4.6 Разметка интерфейса

Интерфейс состоит из поля ввода с кнопкой добавления, трёх кнопок фильтрации, списка задач и счётчика. Каждая задача содержит флажок для

отметки о выполнении и кнопку удаления. Оформление вынесено в файл `styles.css`, листинг которого приведён в приложении А.

Листинг 5 – Файл `index.html`

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Список задач</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <main class="app">
      <h1 class="app__title">Список задач</h1>

      <form class="form" id="task-form" autocomplete="off">
        <input
          class="form__input"
          id="task-input"
          type="text"
          placeholder="Что нужно сделать?"
          maxlength="80">
        <button class="form__button" type="submit">Добавить</button>
      </form>

      <div class="filters">
        <button class="filter" data-filter="all" type="button">Все</button>
        <button class="filter" data-filter="active" type="button">
          Невыполненные
        </button>
        <button class="filter" data-filter="done" type="button">
          Выполненные
        </button>
      </div>

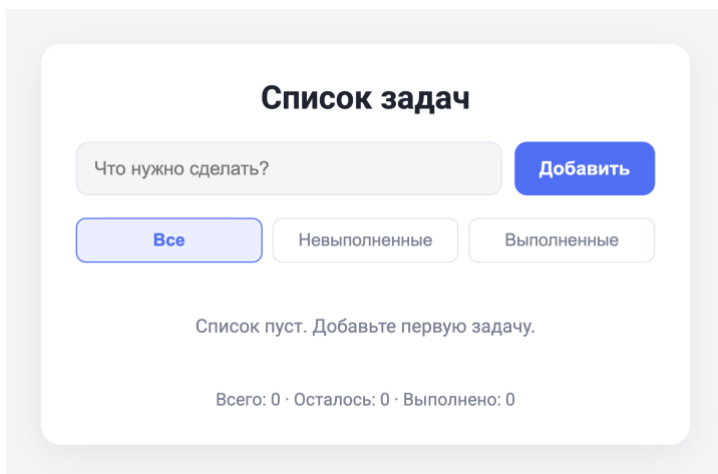
      <ul class="tasks" id="task-list"></ul>

      <p class="counter" id="counter"></p>
    </main>

    <script src="src/taskModel.js"></script>
    <script src="src/store.js"></script>
    <script src="src/view.js"></script>
    <script src="src/app.js"></script>
  </body>
</html>
```

5 РЕЗУЛЬТАТЫ РАБОТЫ ПРИЛОЖЕНИЯ

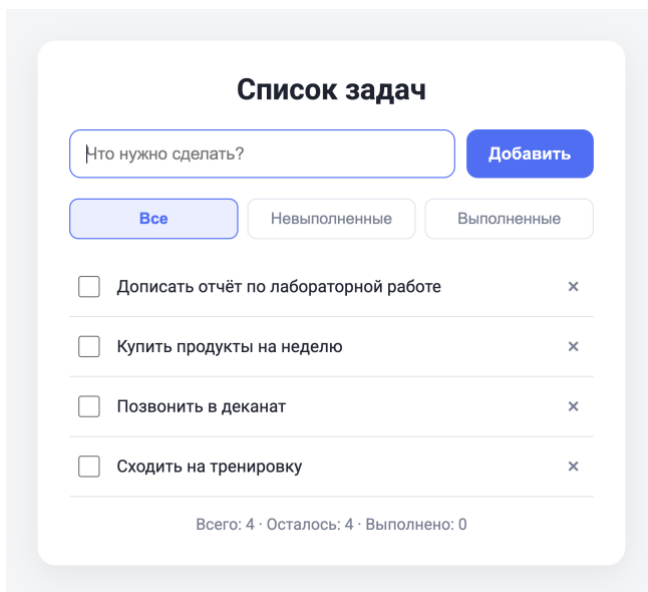
При первом открытии страницы список задач пуст, вместо него отображается подсказка, а счётчик показывает нулевые значения (рисунок 2).



The screenshot shows a web interface titled "Список задач" (Task List). At the top, there is a text input field with the placeholder "Что нужно сделать?" (What needs to be done?) and a blue "Добавить" (Add) button. Below the input field are three filter buttons: "Все" (All), "Невыполненные" (Not completed), and "Выполненные" (Completed). The "Все" button is highlighted. In the center, a message states "Список пуст. Добавьте первую задачу." (List is empty. Add the first task.). At the bottom, a status bar shows "Всего: 0 · Осталось: 0 · Выполнено: 0" (Total: 0 · Remaining: 0 · Completed: 0).

Рисунок 2 – Начальное состояние приложения

Задачи добавляются вводом текста в поле и нажатием кнопки «Добавить» либо клавиши Enter. После добавления четырёх задач список и счётчик обновились (рисунок 3).



The screenshot shows the same web interface as before, but now with a list of four tasks. The input field is empty. The filter buttons remain the same. The task list contains four items, each with a checkbox, a description, and a delete icon (x):

- ☐ Дописать отчёт по лабораторной работе
- ☐ Купить продукты на неделю
- ☐ Позвонить в деканат
- ☐ Сходить на тренировку

The status bar at the bottom now shows "Всего: 4 · Осталось: 4 · Выполнено: 0" (Total: 4 · Remaining: 4 · Completed: 0).

Рисунок 3 – Список после добавления задач

Отметка о выполнении выставляется флажком: выполненные задачи выделяются серым цветом и зачёркиванием, а счётчик пересчитывает количество выполненных и оставшихся задач (рисунок 4).

Список задач

Что нужно сделать? Добавить

Все Невыполненные Выполненные

☐	Дописать отчёт по лабораторной работе	×
☒	Купить продукты на неделю	×
☐	Позвонить в деканат	×
☒	Сходить на тренировку	×

Всего: 4 · Осталось: 2 · Выполнено: 2

Рисунок 4 – Задачи, отмеченные как выполненные

Кнопка «Невыполненные» оставляет в списке только задачи без отметки о выполнении (рисунок 5). При этом сам список задач не изменяется: функция `filterTasks` возвращает новый массив, а счётчик по-прежнему показывает статистику по всем четырём задачам.

Список задач

Что нужно сделать? Добавить

Все Невыполненные Выполненные

☐	Дописать отчёт по лабораторной работе	×
☐	Позвонить в деканат	×

Всего: 4 · Осталось: 2 · Выполнено: 2

Рисунок 5 – Фильтрация: невыполненные задачи

Кнопка «Выполненные» отображает только отмеченные задачи (рисунок 6).

The screenshot shows a web interface titled "Список задач" (Task List). At the top, there is a search bar with the placeholder text "Что нужно сделать?" (What needs to be done?) and a blue "Добавить" (Add) button. Below the search bar are three filter buttons: "Все" (All), "Невыполненные" (Not completed), and "Выполненные" (Completed), with the last one being highlighted. The task list contains two items, both marked as completed with blue checkmarks: "Купить продукты на неделю" (Buy groceries for the week) and "Сходить на тренировку" (Go to the gym). Each item has a small 'x' icon for deletion on the right. At the bottom, a summary bar shows "Всего: 4 · Осталось: 2 · Выполнено: 2" (Total: 4 · Remaining: 2 · Completed: 2).

Рисунок 6 – Фильтрация: выполненные задачи

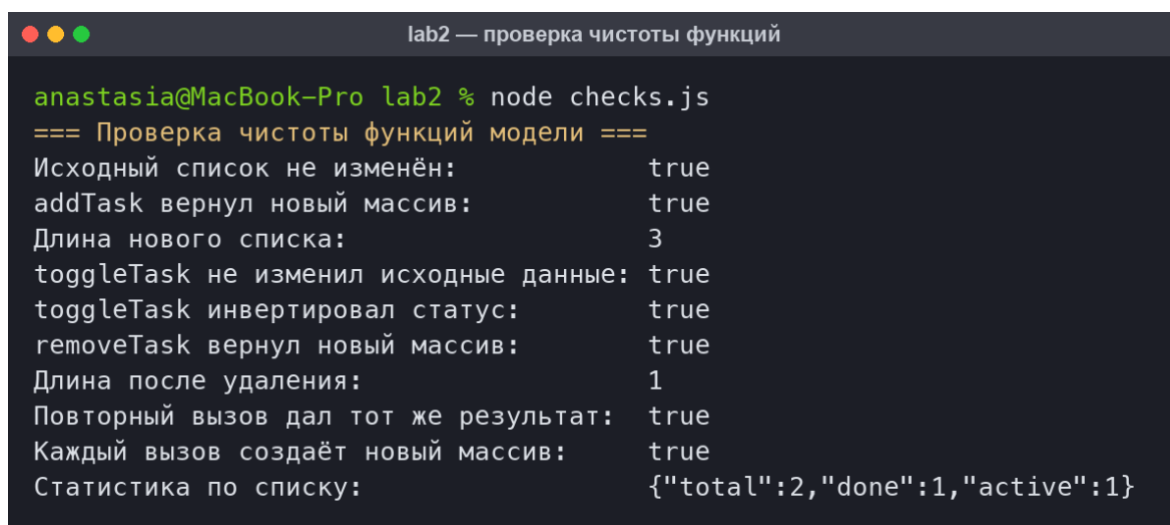
Удаление выполняется кнопкой с крестиком справа от задачи. После удаления задачи «Позвонить в деканат» в списке осталось три задачи, счётчик обновился (рисунок 7).

The screenshot shows the same "Список задач" (Task List) interface. The filter buttons are now "Все" (All), "Невыполненные" (Not completed), and "Выполненные" (Completed), with "Все" being highlighted. The task list now contains three items: "Дописать отчёт по лабораторной работе" (Complete the report on laboratory work), "Купить продукты на неделю" (Buy groceries for the week), and "Сходить на тренировку" (Go to the gym). The first item is unchecked, while the other two are checked. Each item has a small 'x' icon for deletion on the right. At the bottom, the summary bar shows "Всего: 3 · Осталось: 1 · Выполнено: 2" (Total: 3 · Remaining: 1 · Completed: 2).

Рисунок 7 – Список после удаления задачи

6 ПРОВЕРКА ЧИСТОТЫ ФУНКЦИЙ

Для подтверждения иммутабельности данных и чистоты функций модели написан файл `checks.js`, запускаемый командой `node checks.js`. Проверки показали, что после вызовов `addTask`, `toggleTask` и `removeTask` исходный список остаётся неизменным, каждая функция возвращает новый массив, а повторный вызов с теми же аргументами даёт тот же результат (рисунок 8).



```
anastasia@MacBook-Pro lab2 % node checks.js
=== Проверка чистоты функций модели ===
Исходный список не изменён: true
addTask вернул новый массив: true
Длина нового списка: 3
toggleTask не изменил исходные данные: true
toggleTask инвертировал статус: true
removeTask вернул новый массив: true
Длина после удаления: 1
Повторный вызов дал тот же результат: true
Каждый вызов создаёт новый массив: true
Статистика по списку: {"total":2,"done":1,"active":1}
```

Рисунок 8 – Результат проверки чистоты функций модели

7 ССЫЛКА НА РЕПОЗИТОРИЙ

Исходный код лабораторной работы размещён в системе контроля версий Git и доступен по ссылке: <https://github.com/nastenka777/Functional-programming>

Код второй лабораторной работы находится в каталоге `lab2`.

8 ВЫВОДЫ

В ходе выполнения лабораторной работы разработано веб-приложение «Список задач» на HTML, CSS и JavaScript, позволяющее добавлять задачи, отмечать их выполненными, удалять и фильтровать по статусу.

Принципы функционального программирования применены следующим образом. Вся обработка данных выполнена чистыми функциями, собранными в

модуле модели; они не изменяют переданные аргументы и возвращают новые значения. Иммутабельность обеспечена оператором расширения, методами `map` и `filter`, а также методом `Object.freeze` для объектов задач. Функции высшего порядка использованы как встроенные (`map`, `filter`, `reduce`, `forEach`), так и собственные: `createStore` принимает функцию перехода состояний, `subscribe` — обработчик изменений, а предикаты фильтрации передаются в `filter` как данные.

Побочные эффекты изолированы: работа с DOM сосредоточена в функции `render` и в модуле связывания, а изменяемая переменная состояния существует в единственном месте программы. Такое разделение позволило проверить логику приложения отдельно от интерфейса — средствами `Node.js`, без запуска браузера.

ПРИЛОЖЕНИЕ А. Листинг файла стилей

Листинг 6 – Файл styles.css

```
:root {
  --bg: #f4f5f7;
  --card: #ffffff;
  --text: #1f2330;
  --muted: #7b8194;
  --accent: #4f6ef2;
  --accent-soft: #eaeeff;
  --border: #e3e6ee;
  --danger: #e05c5c;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 40px 16px;
  min-height: 100vh;
  font-family: -apple-system, 'Segoe UI', Roboto, Arial, sans-serif;
  font-size: 16px;
  color: var(--text);
  background: var(--bg);
}

.app {
  max-width: 520px;
  margin: 0 auto;
  padding: 28px;
  background: var(--card);
  border-radius: 16px;
  box-shadow: 0 8px 28px rgba(31, 35, 48, 0.08);
}

.app__title {
  margin: 0 0 20px;
  font-size: 26px;
  font-weight: 700;
  text-align: center;
}

.form {
  display: flex;
  gap: 10px;
  margin-bottom: 18px;
}

.form__input {
  flex: 1;
  padding: 12px 14px;
```

```

    font-size: 15px;
    color: var(--text);
    background: var(--bg);
    border: 1px solid var(--border);
    border-radius: 10px;
    outline: none;
}

.form__input:focus {
    border-color: var(--accent);
    background: var(--card);
}

.form__button {
    padding: 12px 20px;
    font-size: 15px;
    font-weight: 600;
    color: #ffffff;
    background: var(--accent);
    border: none;
    border-radius: 10px;
    cursor: pointer;
}

.form__button:hover {
    background: #3f5ce0;
}

.filters {
    display: flex;
    gap: 8px;
    margin-bottom: 16px;
}

.filter {
    flex: 1;
    padding: 9px 10px;
    font-size: 14px;
    color: var(--muted);
    background: transparent;
    border: 1px solid var(--border);
    border-radius: 8px;
    cursor: pointer;
}

.filter:hover {
    color: var(--text);
}

.filter--active {
    color: var(--accent);
    font-weight: 600;
    background: var(--accent-soft);
    border-color: var(--accent);
}

```

```

.task {
  margin: 0;
  padding: 0;
  list-style: none;
}

.task {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 4px;
  border-bottom: 1px solid var(--border);
}

.task__label {
  display: flex;
  align-items: center;
  gap: 12px;
  flex: 1;
  cursor: pointer;
}

.task__check {
  width: 19px;
  height: 19px;
  accent-color: var(--accent);
  cursor: pointer;
}

.task__title {
  font-size: 15px;
  line-height: 1.4;
}

.task--done .task__title {
  color: var(--muted);
  text-decoration: line-through;
}

.task__remove {
  width: 28px;
  height: 28px;
  font-size: 20px;
  line-height: 1;
  color: var(--muted);
  background: transparent;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.task__remove:hover {
  color: var(--danger);
  background: #fdeeee;
}

```

```
.empty {  
  padding: 26px 4px;  
  font-size: 15px;  
  color: var(--muted);  
  text-align: center;  
}
```

```
.counter {  
  margin: 16px 0 0;  
  font-size: 14px;  
  color: var(--muted);  
  text-align: center;  
}
```